

Video Game Programming: Bases of OOP for Games

L03

Prof. Fabio Buttussi, PhD
Human-Computer Interaction Laboratory
Department of Mathematics, Computer Science, and Physics
University of Udine - Italy

Objects in OOP

- Objects in object-oriented programming are the counterparts of real-world objects as well as of real-world living being
- Objects can have properties (**data**) which characterized them as well as actions (**methods**) they can perform
- For example, an object modeling a car can have data about paint color as well as a method to open the trunk
- In some game engines, OOP objects can be attached to game objects in the scene to associate their data and methods
- In Unity, OOP objects are attached to game objects as **components**

Classes

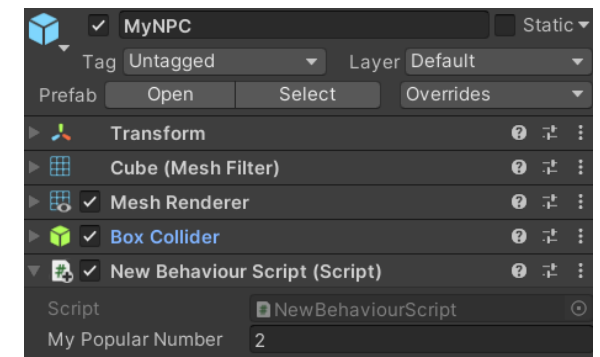
- A class describes data and methods of a type of objects
- Objects are instances of a class
- In C# a class can be defined as in figure

```
public class MyClass  
{  
    ...  
}
```

Data fields

- Objects properties are modeled as class variables called data fields or members
- Data fields can be **private** (accessible only within the class) or **public** (accessible by other classes)
- In Unity, public (or private serializable) fields are also visible and editable using the Inspector in the editor

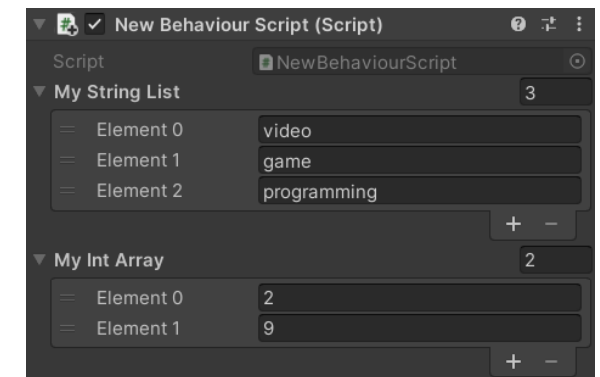
```
public class NewBehaviourScript : MonoBehaviour  
{  
    private int mySegretNumber;  
    public int myPopularNumber = 2;  
}
```



Data structures

- Besides basic types (bool, int, float, char, string,...), data fields can include more complex data structures
- C# supports arrays, lists, dictionaries,...
- List<T> is a generic class that supports lists of different types, e.g., List<string> is a list of strings
- Unity supports editing of different types of data structures

```
public class NewBehaviourScript : MonoBehaviour  
{  
    public List<string> myStringList;  
    public int[] myIntArray;  
}
```



Methods

- Object actions are modeled as class **methods**
- Private methods model internal behaviors of the class
- Public methods allow other classes to activate (call) object actions
- Methods can return nothing (void) or a typed value
- Methods are called using object name followed by dot (.) and method name

```
public class CarBehaviourScript : MonoBehaviour
{
    1 riferimento
    private void CarEngineUpdate()
    {
    }

    0 riferimenti
    public void OpenTrunk()
    {
    }

    0 riferimenti
    public float CalculateCurrentFuelConsumption()
    {
        return deltaKms / deltaFuelLiters;
    }
}
```

Constructor

- Special method to create an object which is a class instance
- Its name / return value is the name of the class
- It can contain statements to initialize fields
- Before using an object of a particular class, it must be initialized using **new** keyword and the constructor
- Basic types are used without initialization
- Unity auto-initializes components of game object, so no explicit constructor is needed
- Values in the Inspector override those in code

```
public class MyClass
{
    private int myIntField;
    private MyClass myOtherClassField;

    0 riferimenti
    public MyClass()
    {
        myIntField = 5;
        myOtherClassField = new MyClass();
    }
}
```

Basic constructs and operations

- C# methods support basic constructs used in imperative languages
 - `if (condition) { ifstatements; } else { elstatements; }`
 - `while (condition) { statements; }`
 - `do { statements; } while (condition);`
 - `for (initializer; condition; iterator) { statements; }`
 - `foreach (type variable in list) { statements; }`
 - `switch (variable) { case value1: statements; case value2: statements; default: defaultstatements; }`
 - ...
- It also support basic mathematical and logical operators: `+`, `-`, `*`, `/`, `==`, `&&`, `||`, `!`,...

Using libraries and creating namespaces

- C# statements can refer to other classes by using libraries
- The full path can be specified at each occurrence, e.g., `UnityEngine.GameObject go = new UnityEngine.GameObject();`
- Or the library can be imported once with **using** directive to have classes and methods directly available
- Classes can be grouped in libraries with **namespace** keyword before defining each class

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

namespace Vehicles
{
    Script Unity | 0 riferimenti
    public class CarBehaviourScript : MonoBehaviour
    {
        1 riferimento
        private void CarEngineUpdate()
        {
            GameObject go = new GameObject();
        }
    }
}
```

Static modifier

- Static modifier allows to specify methods of a class (and whole classes) that are called on the class instead of one of its instances
- For example, `MyStaticClass.MyStaticMethod();`
- Instead of `myDynamicInstance = new MyDynamicClass();`
`myDynamicInstance.DynamicMethod();`
- Video game programmers usually define static classes to make utility libraries, e.g., `MyUtilities.EulerToQuatConverter();`

Inheritance

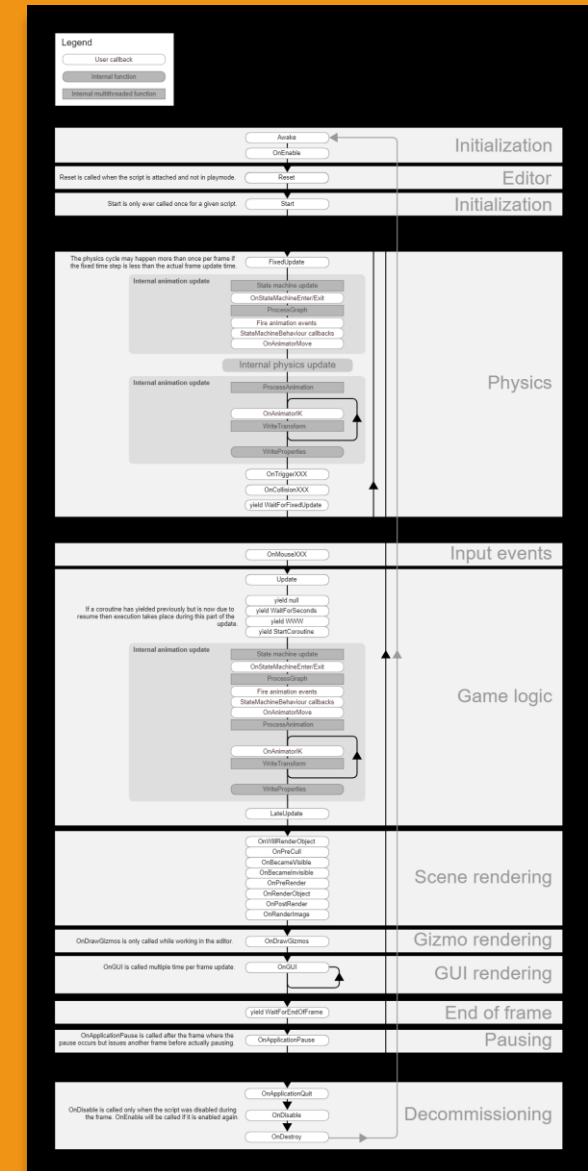
- Classes can be extended using a colon (:)
- For example, `public class Programmer : Person`
- The new class inherits public fields and methods of its base class
- It also inherits **protected** fields and methods, which are not public, but can be seen by derivative classes
- In Unity, components that will be attached to game objects must extend `MonoBehaviour` class

Abstract and virtual modifiers

- A class can have not-implemented methods using the abstract modifier
- In this case, the class cannot be directly used
- Non-abstract derivative classes must implement abstract methods of parent classes
- Methods with virtual modifier are implemented but can be overridden
- Base abstract or virtual methods can be overridden by the new class with the **override** keyword
- Base implementation can be invoked with **base.MethodName()**

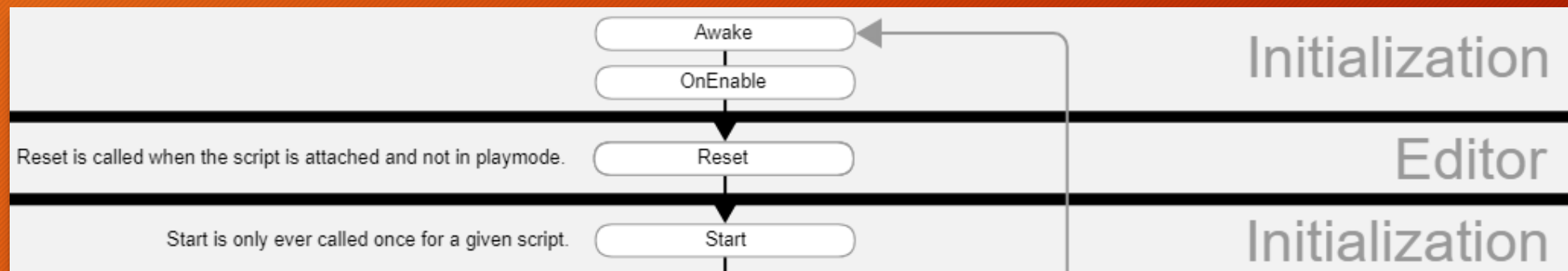
Unity behavior lifecycle

- MonoBehaviour class are automatically instantiated when added to game objects as components
- MonoBehaviour has predefined methods executed in specific moments of the script lifecycle (in fig.)
- Video game programmers can implement those methods and/or add others



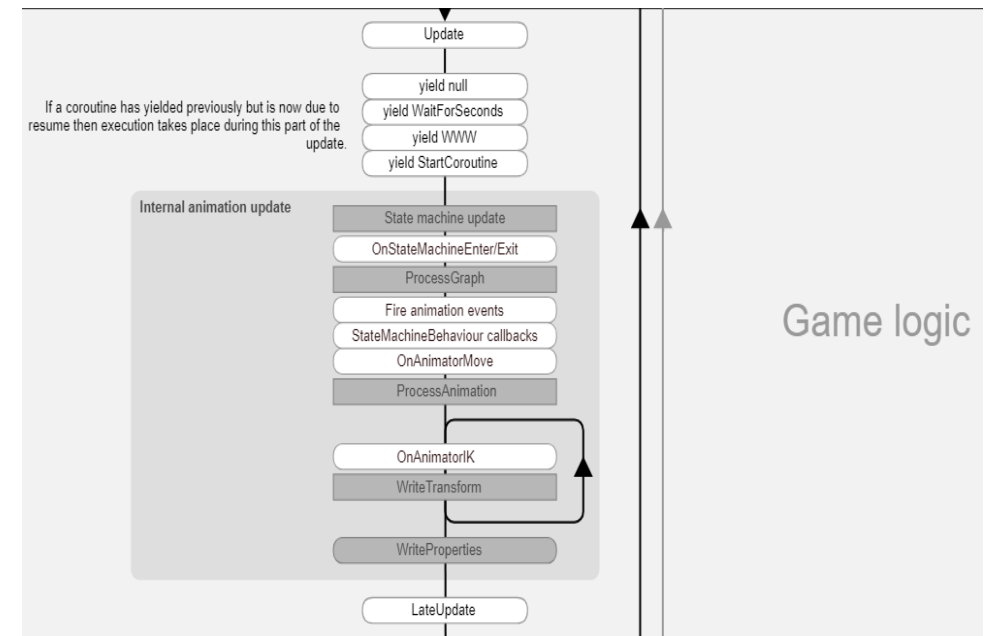
Unity script initialization

- The main initialization method is Start
- Initialization statements that must be done before the component is enabled can be written in Awake



Unity game logic

- Statements inside Update are executed at each frame
- Methods that depend on external events (e.g., waiting for a remote resource) should be implemented in co-routines to avoid blocking the main Update loop
- Statements that must be executed after animation must be written in LateUpdate
- Time elapsed between two frames is available as Time.deltaTime



Access to other components and objects

- In Unity, a component can access to other components on the same object using `GetComponent<Type>()`
- A component can also access components in other game objects, by using `GetComponent` after finding the other objects with:
 - `GameObject.Find("stringname");`
 - `GameObject.FindObjectOfType<Type>()`
 - By having a public field in which the other object is set using the Inspector
 - ...

Is it a script?

- Unity refers to behavior code as scripts
- Scripts are usually pieces of code for interpreted languages
- Typical examples are JavaScript and Bash scripts
- Early versions of Unity supported JavaScript-like scripts (UnityScript)
- UnityScript is deprecated and C# is now the only supported language
- C# is compiled to Common Intermediate Language (CIL), which is then handled by Common Language Runtime (CLR), e.g., .Net or Mono
- Unity has also developed a backend (IL2CPP) to compile to native code
- Nevertheless, Unity programmers write a collection of scripts rather than a full program, which is instead handled by the game engine

Bibliografy

- Hardman, Game Programming with Unity and C#: A Complete Beginner's Guide, Apress, 2020
- **Ferrone, Learning C# by Developing Games with Unity: Get to Grips with Coding in C# and Build Simple 3D Games in Unity 2022 from the Ground up, 7th ed, Packt Publishing, 2022**
- Davis, Unity 3D game development: designed for passionate game developers - engineered to build professional games, Packt Publishing, 2022
- Gomila Salas Juan Gabriel, Borromeo Nicolas Alejandro, Hands-On Unity Game Development, 4th edition, Packt Publishing, 2024

